

00 — Architecture Overview

Status: DRAFT v0.1 (2026-06-12)

Read this first. It is the big picture. The other files are the details.

What the product is

A CPG analytics app. It helps a business user move through four steps:

What happened?	→ Dashboard
Why did it happen?	→ Insights + Drilldown
What do I do next?	→ Recommended Actions
Let me dig in	→ Chat

All four share one user context and one access scope.

The one rule (never break it)

Facts are deterministic.	Numbers come from Cube.js + SQL.
Language is intelligent.	The LLM only writes the words.

The LLM never makes up a number, never sets a score, never bypasses access.

The secret sauce

We ship ready-made **persona cards** (SO, ASM, RSM, NSM, Brand Manager, and more). Each card holds CPG know-how: the role's job, KPIs, insight types, question bank, and sensible defaults. A buyer just assigns each user to a card. Setup is tiny.

How a user is assembled (three tiers)

Persona Card (vendor IP) → Tenant Binding (buyer data + RBAC) → Individual Model (user choice)



Details in 20-resolution.md .

The schemas in this package

- 10 Persona Card → one common shape for every role (the anchor)
- 11 Catalog → the preference/question slots and their defaults
- 12 Metric Registry → each metric, mapped to a Cube.js measure
- 13 Insight Rules → deterministic checks that find risks/opportunities
- 14 Section Grammar → how the home page is laid out
- 20 Resolution → runtime: tiers, RBAC, scope, freshness

How they connect:

Persona Card (10)

- ├─ kpis → Metric Registry (12)
- ├─ insight_types → Insight Rules (13)
- ├─ preferences → Preference Catalog (11)
- ├─ questions → Question Bank (lives on the card; points to 12 + 13)
- └─ home layout → Section Grammar (14)

Key decisions so far

- **One schema per concept, common to all levels.** Differences are field values.
- **Cards are thin.** They reference shared registries (12, 13, 11).
- **Scope = job description.** In the job → answer; outside → decline. (Topic only.)
- **RBAC is a separate, hard fence.** It controls data access, not the LLM.
- **Question bank = answers turned into questions.** Every question has an answer first.
- **Phase 1 = defaults + confirm.** Learning user habits comes later.
- **Depth is capped by result size, not hierarchy level.** Top-N is allowed at any depth.

The flow, end to end

Login → resolve role + RBAC → load card → apply user choices
→ query Cube.js (access-filtered) → run insight rules → score
→ build home (section grammar) → LLM writes the words → store payload
→ render → user asks → scope check → query → narrate → answer

Build order for the team

- | | |
|--|-----------------------------|
| 1. Metric Registry (12) + Catalog (11) | ← lowest-level contracts |
| 2. Persona Card (10) + example cards | ← validate authoring |
| 3. Insight Rules (13) | ← wire deterministic checks |
| 4. Resolution (20) | ← assemble the three tiers |
| 5. Section Grammar (14) | ← render the payload |